



Abstract

Linear Genetic Programming (LGP) is an efficient version of Genetic Programming used since the early 1990s as an alternative representation to expression trees for Genetic Programming tasks. Although Genetic Programming has the potential to self-modify its own code within an evolutionary process, rarely has this been studied systematically in the past. Here we report on an implementation of the concept of self-modification in a system we call Self-Modifying Linear Genetic Programming (SMLGP). The code is evolved using a modified variant of LGP, with the difference that programs are also capable of modifying themselves during their runtime. This opens up new possibilities for arriving at very compact solutions to problems. We study the new approach in the context of self-replication and symbolic regression, distinguishing real self-modification from the related mechanisms of retained memory and program counter looping that our system also exploits.

Design

Self-Modifying Linear Genetic Programming is a new form of GP similar to Self-Modifying Cartesian Genetic Programming (SMCGP) except with the base implementation of LGP instead of CGP (Harding et al., 2007, 2011). Just as SMCGP contains methods to modify its own graph, SMLGP introduces methods to manipulate the memory in which code is stored. Each individual consists of code blocks as shown in Fig. 1. The first memory bank represents the registers found in standard LGP with the second memory bank containing the program itself. Each line of code consists of four values as shown by Fig. 2. Code operates similarly to assembly level code such as PEP9 (Warford, 2016). Tab. 1 shows all defined operation and Tab. 2 depicts the addressing modes. Together, this allows an individual to execute code that can not only modify registers to produce an output, but can also modify and reference its own code.

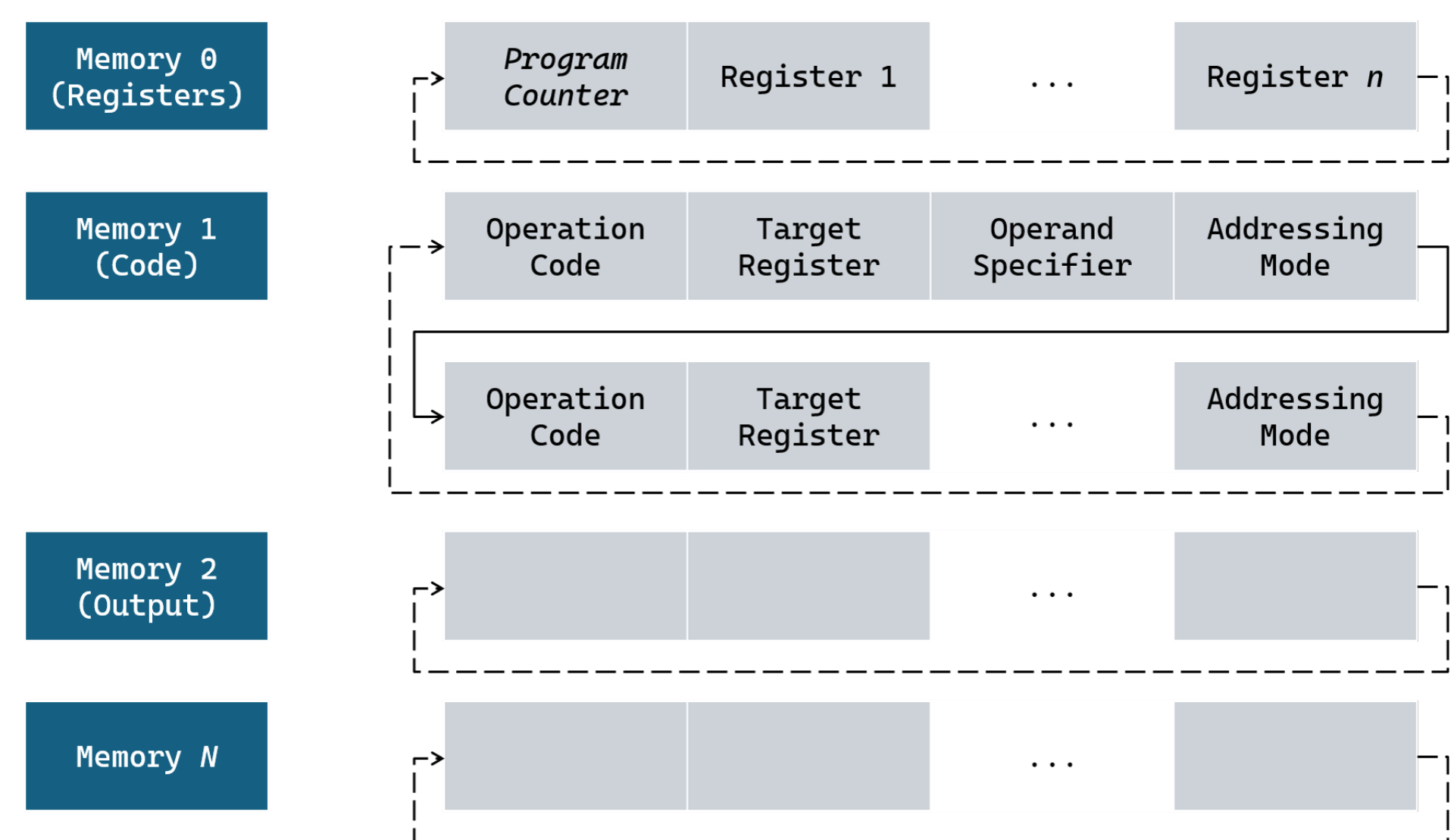


Fig. 1: The available addressing modes used by each line of code where n is the index of a memory bank such that $0 \leq n \leq N - 1$.

Single Line Of Code			
Operation Code	Target Register	Operand Specifier	Addressing Mode
Register		Operand	

Fig. 2: The four values that make up a single line of code.

Operation	Description
STOP	Halt execution
LOAD	Load the operand into the register
STORE	Store the value from the register at the operand
ADD	Add the operand to the register
SUB	Subtract the operand from the register
MUL	Multiply the operand to the register
DIV	Protected divide the operand from the register
IFEQ	If the register is not equal to the operand, increase the PC by an extra line

Tab. 1: All defined operations used by the SMLGP model.

Value	Addressing Mode	Description
0	IMMEDIATE	OP_SPEC
$1 + 2n$	MEMn_DIRECT	MEMn [OP_SPEC]
$2 + 2n$	MEMn_INDIRECT	MEMn [REGS [OP_SPEC]]

Tab. 2: The available addressing modes used by each line of code where n is the index of a memory bank such that $0 \leq n \leq N - 1$.

Self-Reproduction

For this experiment, our goal is to evolve individuals that could copy their code from CODE into the another memory bank named MEM2. Fitness values are calculated by comparing each value between the two memory banks and counting the number of differences. A smaller value represents less errors, and the most fit individual, with value zero, indicates a perfect copy.

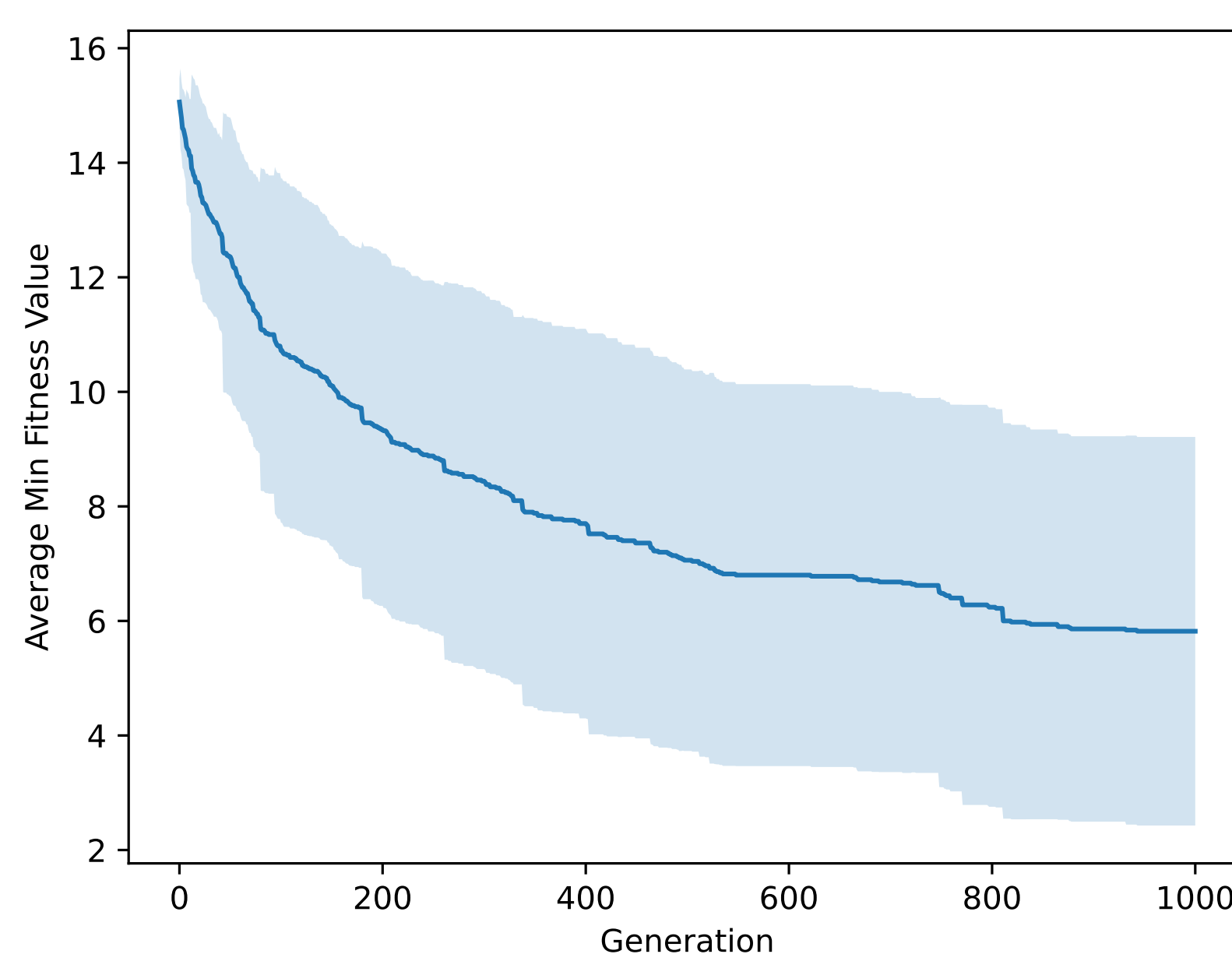


Fig. 3: Evolution for self-replication as shown by the mean of the min fitness for each run.

	CODE				Interpreted Meaning			
0	13	13	11	4	LOAD	1	3	CODE_INDIRECT
4	8	13	3	6	STORE	1	3	MEM2_INDIRECT
8	10	9	8	12	SUB	1	8	MEM2_DIRECT
12	15	11	13	14	ADD	3	1	IMMEDIATE

Tab. 3: An evolved program capable of self-reproduction. The value in register 3 is used as the current index of memory to copy over. Afterwards, register 3 is incremented by 1.

Retained-Memory Evaluation

The goal of this experiment is to calculate the sum of squares which can also be expressed in a closed form:

$$f(n) = \sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6} \quad (1)$$

Memory is either reset or retained. Reset memory operates like usual and code is reset between test cases. Retained memory uses the code from the previous test case and keeps any changes it may have made. Perhaps not surprisingly, allowing for the retention of memory improved results.

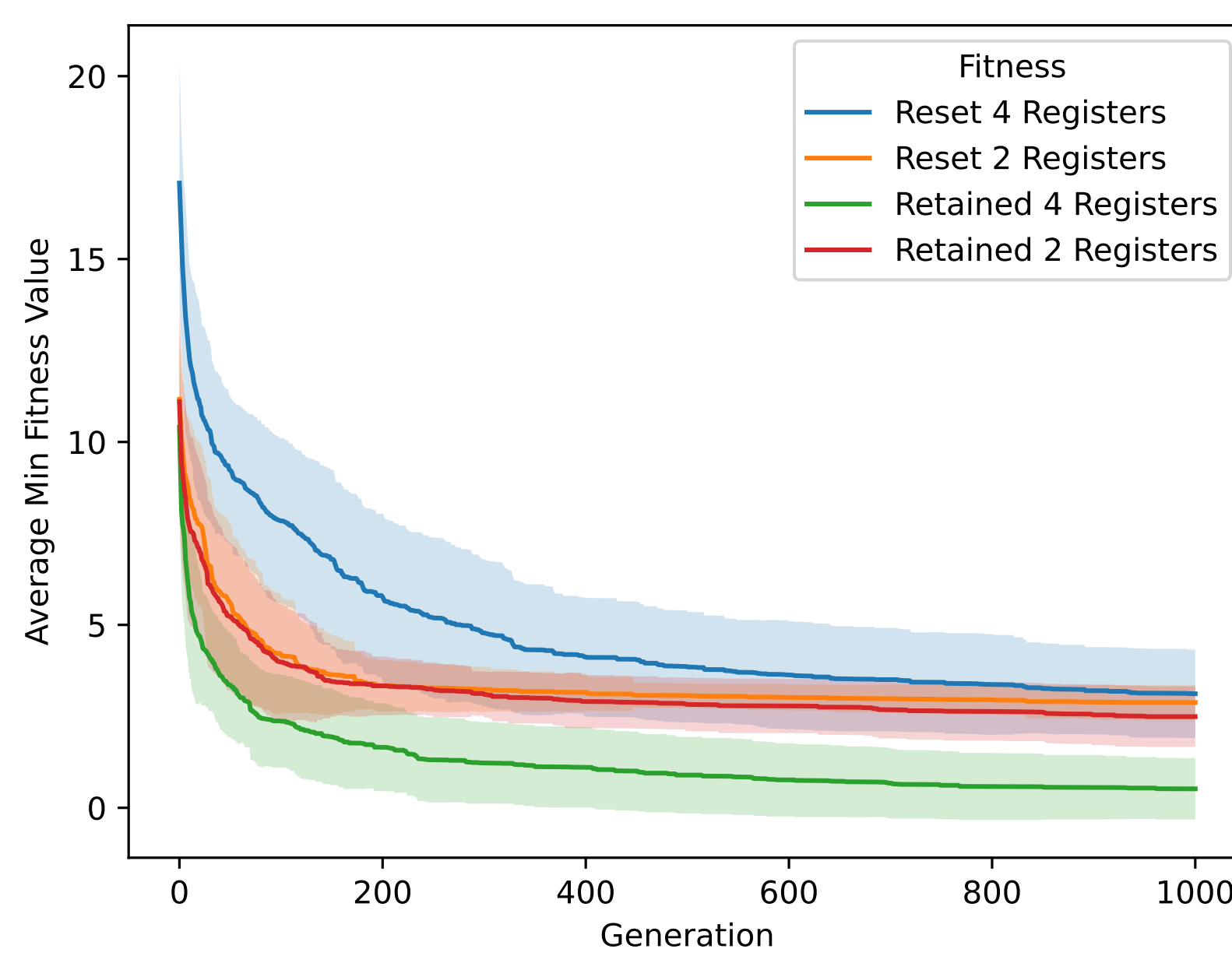


Fig. 4: Evolution for the sum of squares as shown by the mean of the min fitness for each run.

	CODE				Interpreted Meaning			
0	5	1	3	3	MUL	1	1	REGS_DIRECT
4	3	3	8	1	ADD	3	1	REGS_DIRECT
8	0	0	4	3	STOP			
12	3	3	3	6	ADD	3	3	REGS_INDIRECT

Tab. 4: An evolved program to calculate the sum of squares using retained-memory evaluation and 4 registers. The program multiplies the input by itself then adds it to the output stored in register 3.

Self-Modifying Register Allocation

For this experiment, only two registers are allocated. The first register still acts as the PC but the second register must act as both, input and output register. Here, a perfect solution is found that stores the previous result within the code itself which is then used in calculating the rolling sum.

	CODE				Interpreted Meaning			
0	3	0	3	10	ADD	0	0	REGS_INDIRECT
4	5	1	3	8	MUL	1	1	REGS_DIRECT
8	7	0	2	11	IFEQ	0	1	REGS_INDIRECT
12	4	1	2	0	SUB	1	205	IMMEDIATE
16	1	0	4	26	LOAD	0	10	CODE_DIRECT
20	3	1	1	7	ADD	1	1	REGS_INDIRECT
24	3	1	2	18	ADD	1	0	REGS_INDIRECT
28	3	1	3	10	ADD	1	310	IMMEDIATE

Tab. 5: An evolved program to calculate the sum of squares using retained-memory evaluation and 2 registers. Lines starting at 0, 8, and 12 are neutral, the line at 4 is what squares the input. The line at 16 modifies the PC to a value not divisible by 4 and this offsets the values used for each line of code.

PC	CODE	Trace	Interpreted Meaning			
0	3	y_0	310	2	ADD	y_0 0 REGS_INDIRECT
4	5	1	389	1	MUL	1 1 REGS_DIRECT
8	7	0	211	2	IFEQ	0 1 REGS_INDIRECT
16	1	0	426	3	LOAD	0 10 CODE_DIRECT
211	3	3	1	173	ADD	1 1 CODE_DIRECT
215	2	3	1	218	STORE	1 1 CODE_DIRECT
219	2	3	y_1	310	STORE	1 y_1 IMMEDIATE
223	0	3	y_0	310	STOP	1 y_0 IMMEDIATE

Tab. 6: The actual execution trace of the program shown in Tab. 5. y_0 and y_1 are both locations in the code itself where the previous output is stored.

Weighted-Contextual Mutation

This experiment targets the same symbolic regression as before (Eq. 1), except here with limited access to self-modification. The only remaining feature allowed is that code would still repeat until 16 lines are executed and the program times out. This experiment was designed to demonstrate how assigning weights to certain values could adjust the likelihood of self modification. In the end, a perfect solution was found for the given domain that uses the looping of code to compress the mathematical expression down to 1 neutral and 4 functional lines of code. The result still relies heavily on the design of SMLGP, even with the given limitations.

	Internal Value				Translated Meaning			
0	2	2	1	1	ADD	2	1	REGS_DIRECT
4	5	2	1	0	DIV	2	1	IMMEDIATE
8	2	3	2	1	ADD	3	2	REGS_DIRECT
12	5	3	3	0	DIV	3	3	IMMEDIATE
16	4	2	1	1	MUL	2	1	REGS_DIRECT

Tab. 7: An evolved program to calculate the sum of squares using reset-memory evaluation and 4 registers. Self modification was limited but the solution still uses the looping of the PC to repeat lines of code until 16 lines have been executed. This operates as a method of compressing code and yields an exact solution under the domain of the problem.

References

- Harding, S., Miller, J., and Banzhaf, W. (2007). Self-modifying Cartesian Genetic Programming. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation*, pages 1021–1028, New York, NY, USA. ACM Press.
- Harding, S., Miller, J., and Banzhaf, W. (2011). Self-Modifying Cartesian Genetic Programming. In Miller, J., editor, *Cartesian Genetic Programming*, pages 101–124. Springer, Berlin, Heidelberg, Germany.
- Warford, J. S. (2016). *Computer Systems*. Jones and Bartlett, Burlington, MA, USA, 5th edition.